

Assignment 3 - 2025

The dataset can be downloaded from [here](https://myuni.adelaide.edu.au/courses/101197/files/16524286/download).

(<https://myuni.adelaide.edu.au/courses/101197/files/16524286/download>).

1. Wine Quality Prediction with 1NN (K-d Tree)

Wine experts evaluate the quality of wine based on sensory data. We could also collect the features of wine from objective tests, thus, the objective features (simply called features hereafter) could be used to predict the expert's judgment, which is the quality rating of the wine. This could be formed as a supervised learning problem by using the features as the data and the wine quality rating as the corresponding labels.

In this assignment, we provide features obtained from physicochemical statistics for each white wine sample and its corresponding rating provided by wine experts. You are expected to implement the k-d tree (KDT) and use the training set to train your k-d tree, then provide wine quality prediction on the test set by searching the tree

Wine quality rating is measured in the range of 0-9. In our dataset, we only keep the samples for quality ratings 5, 6, and 7. The 11 features are listed as follows [1]:

- **f_acid**: fixed acidity
- **v_acid**: volatile acidity
- **c_acid**: citric acid
- **res_sugar**: residual sugar
- **chlorides**: chlorides
- **fs_dioxide**: free sulfur dioxide
- **ts_dioxide**: total sulfur dioxide
- **density**: density
- **pH**: pH
- **sulphates**: sulphates
- **alcohol**: alcohol

Explanation of the Data.

train: The first 11 columns represent the 11 features and the 12th column is the wine quality. A sample of the data is depicted as follows:

f_acid	v_acid	c_acid	res_sugar	chlorides	fs_dioxide	ts_dioxide	density	pH	sulphates	alcohol	quality
8.10	0.270	0.41	1.45	0.033	11.0	63.0	0.99080	2.99	0.56	12.0	5
8.60	0.230	0.40	4.20	0.035	17.0	109.0	0.99470	3.14	0.53	9.7	5
7.90	0.180	0.74	1.20	0.040	16.0	75.0	0.99200	3.18	0.63	10.8	5
8.30	0.420	0.62	19.25	0.040	41.0	172.0	1.00020	2.98	0.67	9.7	5

6.50	0.310	0.14	7.50	0.044	34.0	133.0	0.99550	3.22	0.50	9.5	5
------	-------	------	------	-------	------	-------	---------	------	------	-----	---

test: The 12th column `wine quality` is not given to the test samples. A sample of the data is depicted as follows:

<code>f_acid</code>	<code>v_acid</code>	<code>c_acid</code>	<code>res_sugar</code>	<code>chlorides</code>	<code>fs_dioxide</code>	<code>ts_dioxide</code>	<code>density</code>	<code>pH</code>	<code>sulphates</code>	<code>alcohol</code>
7.0	0.360	0.14	11.60	0.043	35.0	228.0	0.99770	3.13	0.51	8.900000
6.3	0.270	0.18	7.70	0.048	45.0	186.0	0.99620	3.23	0.47	9.000000
7.2	0.290	0.20	7.70	0.046	51.0	174.0	0.99582	3.16	0.52	9.500000
7.1	0.140	0.35	1.40	0.039	24.0	128.0	0.99212	2.97	0.68	10.400000
7.6	0.480	0.28	10.40	0.049	57.0	205.0	0.99748	3.24	0.45	9.300000

1.1. 1NN (K-d Tree)

From the given training data, our goal is to learn a function that can predict the wine quality rating of a wine sample, based on the features. In this assignment, the predictor function will be constructed as a k-d tree. Since the features are continuously valued, you shall apply the k-d tree algorithm for continuous data, as outlined in Algorithm 1. It is the same as taught in the lecture. Once the tree is constructed, you will search the tree to find the 1-nearest neighbour of a query point and label the query point. Please refer to the search logic taught in the lecture to write your code for the 1NN search.

Algorithm 1 BuildKdTree(P , D)

Require: A set of points P of M dimensions and current depth D .

```

1: if  $P$  is empty then
2:   return null
3: else if  $P$  only has one data point then
4:   Create new node node
5:    $node.d \leftarrow d$ 
6:    $node.val \leftarrow val$ 
7:    $node.point \leftarrow current\ point$ 
8:   return node
9: else
10:   $d \leftarrow D \bmod M$ 
11:   $val \leftarrow$  Median value along dimension among points in  $P$ .
12:  Create new node node.
13:   $node.d \leftarrow d$ 
14:   $node.val \leftarrow val$ 
15:   $node.point \leftarrow$  point at the median along dimension  $d$ 
16:   $node.left \leftarrow$  BuildKdTree(points in  $P$  for which value at dimension  $d$  is less than or equal to  $val$ ,  $D+1$ )
17:   $node.right \leftarrow$  BuildKdTree(points in  $P$  for which value at dimension  $d$  is equal to or greater than  $val$ ,  $D+1$ )
18:  return node
19: end if

```

Sorting (at each level) may not be necessary depending on your implementation. Google/ChatGPT for "Quickselect" if you are interested in implementing a K-d tree without doing the sorting. Full pre-sorting

before building the k-d tree is another possible solution, but it is usually more complex.

For students whose student ID ends with an odd number, please implement Algorithm 1 with the yellow colored "or equal to" in line 16; you should ignore the blue colored "equal to or" in line 17.

Likewise, for students whose student ID ends with an even number, please implement Algorithm 1 with the blue colored "equal to or" in line 17; you should ignore the yellow colored "or equal to" in line 16.

We give you some hints to help you investigate if you are stuck or your implementation does not pass the Gradescope tests.

1. What is the median of an even number of values, say [1,2,3,4]? The median is 2.5. Does this surprise you? Perhaps verify it by using `numpy.median()`. This may potentially be a problem in implementing the median part in your Algorithm 1, depending on the implementation.
2. Have you checked how your implementation behaves when there is more than one value that is equal to the median?
3. Have you checked how your implementation behaves in extreme cases, such as when there are only two points or three points left to split? Why would your implementation run into an infinite loop by following Algorithm 1?

1.3. Deliverable

Write your k-d tree program in Python 3.10.6 in a file called **nn_kdtree.py**. Similar to the previous assignments, in your `nn_kdtree.py`, please define the two constants below; we will import them to correctly calculate your mark for the UG/PG variations and to determine the ID-based variation of Algorithm 1.

```
STUDENT_ID = 'a1234567' # your student ID
DEGREE = 'UG' # or PG if you are in the postgraduate course
```

Your program must be able to run as follows:

```
python3 nn_kdtree.py [train] [test] [dimension]
```

The inputs/options to the program are as follows:

- `[train]` specifies the path to a file containing a set of training data
- `[test]` specifies the path to a file containing a set of testing data
- `[dimension]` is used to decide which dimension to start the comparison. (Algorithm 1)

Given the inputs, your program must construct a k-d tree (following the prescribed algorithms) using the training data, then predict the quality rating of each of the wine samples in the testing data.

Your program must print two sections of content to standard output (i.e., the command prompt):

1. The number of points in the left and right trees after the first split of the k-d tree. An example format is:

```
....l487
....r412
```

The number of '.' to print is indicated by D in Algorithm 1, here D = 4 in this example. 'l' and 'r' denote the left and right subtrees, respectively. The numbers indicate the number of points in each subtree.

2. The list of predicted wine quality ratings, vertically based on the order in which the testing cases appear in `[test]`. An example format is in `sample_data/test-sample-result`.

```
5
5
5
```

To summarise, the full expected output format would be:

```
....l487
....r412
5
5
5
```

We will match both output sections to our expected result (i.e., ground truth). We match the point split result because choosing a different D will result in a different split, but the prediction does not necessarily change. If your output does not match the expected, we will not show you the correct prediction. We will only show you the first-level tree split result to help you debug.

1.4. Python Libraries

You are allowed to use the Python standard library to write your k-d tree learning program (see <https://docs.python.org/3/library>  <https://docs.python.org/3/library>) for the components that make up the Python v3.10.6 standard library). In addition to the standard library, you are allowed to use NumPy and Pandas. Note that the marking program will not be able to run your program to completion if other third-party libraries are used. **You are NOT allowed to use implemented tree structures from any Python package, otherwise, the mark will be set to 0.**

1.4. Submission

You must submit your program files on the Gradescope portal linked in MyUni.

1.5. Expected Run Time

Your program must be able to terminate within 600 seconds on the sample data given.

1.6. Debugging Suggestions

Step-by-step debugging by checking intermediate values/results will help you identify the problems with your code. This function is enabled by most Python IDEs. If not in your case, you could also print the intermediate values out. You could use sample data or create data in the same format for debugging

1.7. Assessment

Gradescope will compile and run your code on several test problems. If it passes all tests, you will get **10% (undergrads)** or **8% (postgrads)** of the overall course mark.

There will be no further manual inspection/grading of your program to award marks based on coding style, commenting, or “amount” of code written. However, we will inspect your submission should Gradescope flag your submission with high code similarity to other submissions.

1.8. Using other source code

You may not use other source code for this assignment. All submitted code must be your own work written from scratch. Only by writing the solution yourself will you fully understand the concept.

1.9. Due date and late submission policy

This assessment's due date is specified on MyUni. The usual late penalty applies. If you need an extension, please apply as soon as possible and apply directly via the [assessment extension form \(https://login.adelaide.edu.au/cas/login?service=https%3A//www.adelaide.edu.au/student/exams/casservice%3Freturnto%3Dstudent/exams/assessment-applications/assessment-extension\)](https://login.adelaide.edu.au/cas/login?service=https%3A//www.adelaide.edu.au/student/exams/casservice%3Freturnto%3Dstudent/exams/assessment-applications/assessment-extension). The assessment information can be found [here \(https://myuni.adelaide.edu.au/courses/101197/pages/assessment-overview?module_item_id=3601266\)](https://myuni.adelaide.edu.au/courses/101197/pages/assessment-overview?module_item_id=3601266).

2. Wine Quality Prediction with Random Forest

For **postgraduate** students, completing this section will give you the remaining **2%** of the assignment marks. In this task, you will extend your knowledge learned from the k-d tree to make a k-d forest. The process for a simplified k-d forest given N input-output pairs is:

1. Randomly select a set of N' distinct samples (i.e., no duplicates) where $N' = N * 80\%$ (round to integer). This dataset is used for constructing a k-d tree (i.e., the root node of the k-d tree)
2. Build a k-d tree on the dataset from (1) and apply Algorithm 1.
3. Repeat (1) and (2) until reaching the maximum number of trees.

This process is also shown in **Algorithm 2**. In k-d forest learning, a sample set is used to construct a k-d tree. That is to say, different trees in the forest could have different root data. For prediction, the k-d forest will choose the most voted label as its prediction. For the wine quality prediction task, you shall apply **Algorithm 2** for k-d forest learning and apply **Algorithm 3** to predict the wine quality for a new wine sample. To generate samples, please use the following function to generate the same samples as our testing scripts.

```
import random
def generate_sample_indexes(N: int, N_prime: int, n_trees: int, rand_seed: int) -> List[int]:
    index_list = [i for i in range(0, N)] # create a list of indexes for all data
    sample_indexes = []
    for j in range(0, n_trees):
        random.seed(rand_seed + j) # random_seed is one of the input parameters
        subsample_idx = random.sample(index_list, k=N_prime) # create unique N' indices
        sample_indexes = sample_indexes + subsample_idx
    return sample_indexes
```

Algorithm 2 KdForest($data$, d_list , $rand_seed$)

Require: $data$ in the form of N input-output pairs $\{x_i, y_i\}_{i=1}^N$, d_list a list of depth

- 1: $forest \leftarrow []$
- 2: $n_trees \leftarrow \text{len}(d_list)$
- 3: $sample_indexes \leftarrow N * n_trees$ integers with value in $[0, N)$ generated by using above $generate_sample_indexes()$ function
- 4: $count \leftarrow 0$
- 5: **for** $count < n_trees$ **do**
- 6: $sampld_data \leftarrow N'$ data pairs selected by N' indexes from $sample_indexes$ sequentially
- 7: $n = \text{BuildKdTree}(sampld_data, d_list[count]) \Rightarrow \text{Algorithm 1}$
- 8: $forest.append(n)$

```
9: end for
10: return forest
```

Algorithm 3 Predict_KdForest(*forest*, *data*)

Require: *forest* is a list of tree roots, *data* in the form of attribute values *x*.

```
1: labels ← []
2: for Each tree n in the forest do
3:   label ← 1NN search on tree n
4:   labels.append(n)
5: end for
6: return the most voted label in labels
```

2.1 Deliverables

Write your random forest program in Python 3.10.6 in a file called **nn_kdforest.py**. Your program must be able to run as follows

```
python3 nn_kdforest.py [train] [test] [random_seed] [d_list]
```

The inputs/options to the program are as follows:

- [train] specifies the path to a file containing a set of training data
- [test] specifies the path to a file containing a set of testing data
- [random_seed] is the seed value to be used in the random number generator.
- [d_list] is a list of depth values written in the form of a string such as "[1,2,3]", with **no space** after each comma. In **Algorithm 2**, `n_trees==len(d_list)`.

Given the inputs, your program must learn a random forest (following the prescribed algorithms) using the training data, then predict the quality rating of each wine sample in the testing data.

Similarly to the undergraduate test, your program must **print to standard output**: 1) the first split result of the k-d trees (here the example output corresponds to `d_list = "[1,2,3]"`) in the forest; and 2) the list of predicted wine quality ratings, vertically based on the order in which the testing cases appear in [test]:

```
.l455
.r444
..l436
..r463
...l417
...r482
5
5
5
```

Submit your program in the same way as the submission for Sec. 1. For postgraduates, please submit **nn_kdtree.py** and **nn_kdforest.py**. The due date, late submission policy, and code reuse policy are also the same as in Sec. 1.

2.2 Expected Run Time

Your program must be able to terminate within 600 seconds on the sample data given.

2.3 Debugging Suggestions

In addition to Sec. 1.6, another value worth checking when debugging is (but not limited to):

- the *sample_indexes* – by setting a random seed, the indexes should be the same each time you run the code

2.4 Assessment

Gradescope will compile and run your code on several test problems. If it passes all tests, you will get **2%** of the overall course mark.

References

[1] Cortez, P., Cerdeira, A., Almeida, F., Matos, T., and Reis, J. Modeling wine preferences by data mining from physicochemical properties. *Decision support systems* 47, 4 (2009), 547–553.